

Owner Auth & Onboarding — Design

MVP step 2: authentication, Gate 1 (identity) and Gate 2 (property ownership)

Design for owner sign-up, sign-in, and the two-gate onboarding flow defined in the project memory: Stripe Connect Custom identity verification (Gate 1, required before any listing or payout) and property-ownership verification (Gate 2, required only to unlock Public Mode). Code is not yet implemented — this document defines the schema, flows, and routes to build against.

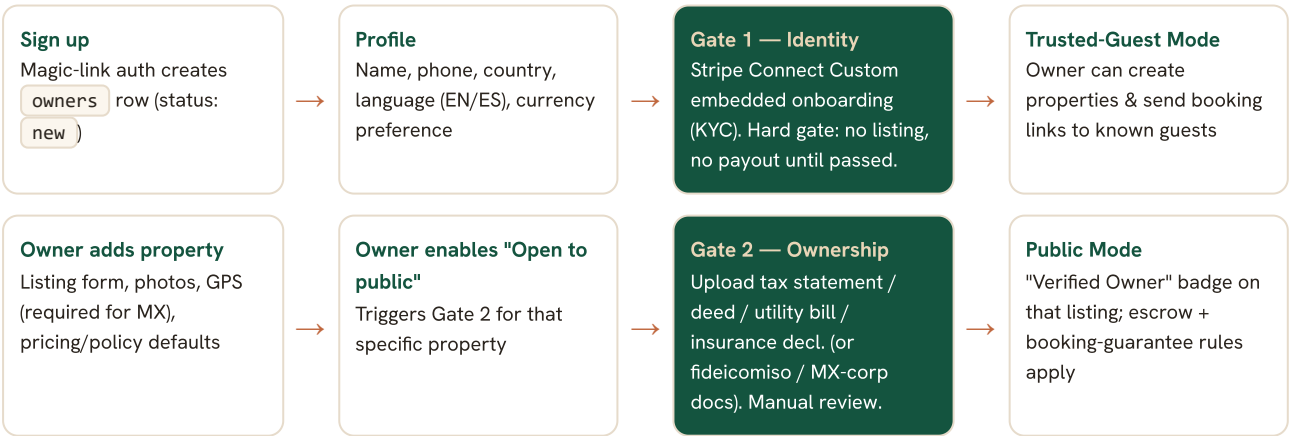
1 • Authentication Flow

Supabase Auth, email magic-link only (no passwords). Session refresh handled by Next.js middleware on every request.



Routes: `/login` (public), `/auth/callback` (public, route handler), `/onboarding/*` (authenticated, Gate-1 incomplete), `/dashboard/*` (authenticated, Gate-1 complete). `middleware.ts` refreshes the Supabase session cookie and redirects unauthenticated users to `/login`.

2 • Owner Onboarding Flow — Two Gates



Key rule: Gate 1 is per *owner* (one Stripe Connect account). Gate 2 is per *property* — an owner with five properties can have some in Trusted-Guest Mode and others in Public Mode independently, since the ownership document must match each property's address/coordinates.

3 • Database Schema (Supabase / Postgres)

All tables use Postgres `uuid` primary keys (`gen_random_uuid()`) and `created_at` / `updated_at` timestamps (`timestampz` , default `now()`). Row-Level Security (RLS) is enabled on every table; policies are summarized per table.

owners

One row per authenticated user. Created by a trigger on `auth.users` insert.

Column	Type	Notes
<code>id</code>	uuid, PK	= <code>auth.users.id</code>
<code>email</code>	text	mirrored from auth for convenience/search
<code>full_name</code>	text	nullable until profile step
<code>phone</code>	text	nullable
<code>country</code>	text	owner's home country (US/CA/etc.)
<code>locale</code>	text	<code>en</code> <code>es</code> , default <code>en</code>
<code>payout_currency</code>	text	default <code>USD</code> ; <code>CAD</code> / <code>MXN</code> selectable later
<code>onboarding_status</code>	text	<code>new</code> <code>profile_complete</code> <code>gate1_complete</code>
<code>stripe_account_id</code>	text	Stripe Connect Custom account id, nullable
<code>stripe_onboarding_status</code>	text	<code>not_started</code> <code>pending</code> <code>verified</code> <code>restricted</code>
<code>stripe_requirements_due</code>	jsonb	cached <code>requirements.currently_due</code> for support visibility
<code>created_at</code> , <code>updated_at</code>	timestampz	

RLS: owner can `select` / `update` only their own row (`id = auth.uid()`). Service-role only for `insert` (via trigger) and for writing `stripe_*` columns from webhooks.

properties

Minimal stub sufficient to support Gate 2 and the onboarding flow; full listing fields (pricing, policies, amenities) are out of scope for this design and land with the Property Creation step.

Column	Type	Notes
<code>id</code>	uuid, PK	
<code>owner_id</code>	uuid, FK → <code>owners.id</code>	
<code>name</code>	text	working title, e.g. "Casa Azul — Todos Santos"
<code>country</code>	text	<code>US</code> <code>MX</code>
<code>address</code>	text	nullable for MX
<code>latitude</code> , <code>longitude</code>	numeric	required when <code>country = 'MX'</code>
<code>mode</code>	text	<code>trusted_guest</code> <code>public</code> , default <code>trusted_guest</code>
<code>created_at</code> , <code>updated_at</code>	timestampz	

RLS: owner can CRUD rows where `owner_id = auth.uid()`. Public booking-page reads happen through a separate `select` - only policy scoped by listing id (added when the booking page is built) — not required for this step.

ownership_verifications (Gate 2)

One row per Gate-2 submission for a property. A property may have multiple rows if a submission is rejected and resubmitted.

Column	Type	Notes
<code>id</code>	uuid, PK	
<code>property_id</code>	uuid, FK → <code>properties.id</code>	
<code>owner_id</code>	uuid, FK → <code>owners.id</code>	denormalized for RLS simplicity
<code>document_type</code>	text	<code>property_tax_statement</code> <code>deed</code> <code>utility_bill</code> <code>insurance_declaration</code> <code>fideicomiso</code> <code>mx_corp_docs</code>
<code>storage_path</code>	text	path in <code>ownership-documents</code> bucket
<code>status</code>	text	<code>pending</code> <code>approved</code> <code>rejected</code> , default <code>pending</code>
<code>reviewer_notes</code>	text	nullable, set on reject
<code>reviewed_by</code>	uuid	nullable, admin user id
<code>reviewed_at</code>	timestamptz	nullable
<code>submitted_at</code>	timestamptz	default <code>now()</code>

RLS: owner can `insert` / `select` their own rows (`owner_id = auth.uid()`); only service-role (admin console) can `update` status/review fields.

Storage buckets

Bucket	Access	Contents
<code>ownership-documents</code>	private — owner can upload/read own files; service-role for admin review	Gate-2 documents (PII — tax statements, deeds, fideicomiso)
<code>property-photos</code>	public read, authenticated write (owner-scoped folder)	Listing photos

Compliance note: no SSNs or government IDs are stored anywhere in this schema — Stripe Connect's embedded components handle that PII directly (per `CLAUDE.md` hard constraint #3). The `ownership-documents` bucket holds property-ownership PII only and is access-controlled accordingly.

4 • Application Routes

Route	Auth	Purpose
/login	public	Email entry, sends magic link
/auth/callback	public	Exchanges magic-link code for session
/onboarding/profile	owner, pre-Gate1	Name, phone, country, locale, currency
/onboarding/verify	owner, pre-Gate1	Embeds Stripe Connect Custom onboarding component (Gate 1)
/dashboard	owner, Gate1 complete	Landing — properties list, status summary
/dashboard/properties/new	owner, Gate1 complete	Create property (stub fields per schema above)
/dashboard/properties/[id]/verify	owner, Gate1 complete	Upload Gate-2 ownership document, shows review status

middleware.ts enforces: unauthenticated → /login ; authenticated + onboarding_status != 'gate1_complete' → /onboarding/* ; authenticated + Gate-1 complete visiting /onboarding/* → redirect to /dashboard .

5 • Coverage Check Against Required Features

Requirement (from project memory)	Covered by this design
Email + magic-link auth (Supabase Auth)	✓ Section 1
Gate 1 — Stripe Connect Custom embedded KYC, hard gate before listing/payout	✓ owners.stripe_* columns + /onboarding/verify ; middleware blocks /dashboard until gate1_complete
No SSNs/ID documents stored on our servers	✓ Stripe-hosted components only; schema has no PII fields for Gate 1
Trusted-Guest Mode unlocked on Gate-1 pass	✓ onboarding_status = 'gate1_complete' unlocks /dashboard + property creation
Gate 2 — property ownership verification, per listing, manual review	✓ ownership_verifications table + /dashboard/properties/[id]/verify , status pending/approved/rejected
Gate 2 accepts US docs (tax statement, deed, utility bill, insurance) AND MX (fideicomiso, MX-corp docs)	✓ document_type enum covers both
Public Mode unlocked per-property on Gate-2 pass	✓ properties.mode flips trusted_guest → public on approved
GPS coordinates required for Mexico properties	✓ properties.latitude/longitude , required when country = 'MX'
Bilingual (EN/ES) support	✓ owners.locale drives UI language from first profile step
Multi-currency payout preference captured early	✓ owners.payout_currency (USD default, CAD/MXN selectable)
RLS — owners see only their own data	✓ policies defined per table

Caretaker seats (5 per plan)	<i>Not in this design</i> — deferred; will extend <code>owners</code> / <code>properties</code> with a <code>caretakers</code> join table when the caretaker portal is built
Property listing fields (pricing, policies, amenities)	<i>Not in this design</i> — <code>properties</code> is intentionally a stub; full listing schema arrives with the Property Creation step (MVP step 3)

This design covers MVP build-sequence step 2 (owner auth and onboarding) in full, including both onboarding gates. Caretaker access and full property/listing fields are explicitly out of scope and noted for their respective later steps.